

1. Magic Methods in Python

- A magic method also called Dunder Method it is a special method in Python whose name starts and ends with double underscores (`__name__`). It allows us to define how an object behaves when Python performs built-in operations on it.
- Magic methods are special methods in Python that are automatically called by Python when we perform certain operations on an object.

2. Why Do We Use Magic Methods?

- We use magic methods to tell Python how our objects should behave when we use built-in operations like `print()`, `+`, `==`, and `len()`.
- `p1 + p2`
- Python doesn't automatically know how to add two Product objects.
- `__add__()`
- we can tell Python what should happen when two Product objects are added.

3. Commonly Used Magic Methods

1) `__init__()`

- Used to initialize an object when it is created.

2) `__str__()`

- Used to define what should be displayed when we use `print(object)`.

3) `__len__()`

- Used to define what `len(object)` should return.

4) `__eq__()`

- Used to define how two objects should be compared using `==`.

5) `__add__()`

- Used to define the behavior of the + operator between objects.

6) `__sub__()`

- Used to define the behavior of the - operator between objects.

7) `__mul__()`

- Used to define the behavior of the * operator between objects.

8) `__truediv__()`

- Used to define the behavior of the / operator between objects.

4. Example :

```
class Addition:

    def __init__(self, num1):
        self.numone = num1

    # print(object)
    def __str__(self):
        return str(self.numone)

    # object1 == object2
    def __eq__(self, other):
        return self.numone == other.numone

    # object1 + object2
    def __add__(self, other):
        return self.numone + other.numone

    # object1 - object2
    def __sub__(self, other):
        return self.numone - other.numone

    # object1 * object2
    def __mul__(self, other):
        return self.numone * other.numone
```

```
# object1 / object2
def __truediv__(self, other):
    return self.numone / other.numone

# object1 % object2
def __mod__(self, other):
    return self.numone % other.numone

n1 = Addition(20)
n2 = Addition(10)

print(n1)          # __str__()
print(n2)          # __str__()

print(n1 == n2)    # __eq__()

print(n1 + n2)     # __add__()
print(n1 - n2)     # __sub__()
print(n1 * n2)     # __mul__()
print(n1 / n2)     # __truediv__()
print(n1 % n2)     # __mod__()
```